합의 프로세스

소개

합의 프로세스는 Oasis Core 네트워크에서 모든 노드가 동일한 블록체인 상태를 유지하도록 보장하는 핵심 메커니즘입니다. 이 프로세스는 CometBFT (이전 Tendermint) 합의 알고리즘을 기반으로 구현되어 있으며, 비잔틴 장애 허용(Byzantine Fault Tolerance) 특성을 제공합니다.

주요 특징

- **비잔틴 장애 허용**: 네트워크의 1/3 미만의 악의적 노드가 있어도 정상 동작
- **즉시 완결성**: 블록이 커밋되면 즉시 최종 확정되어 롤백 불가능
- **높은 처리량**: 초당 수천 건의 트랜잭션 처리 가능
- **결정론적 실행**: 모든 노드에서 동일한 순서로 트랜잭션 실행

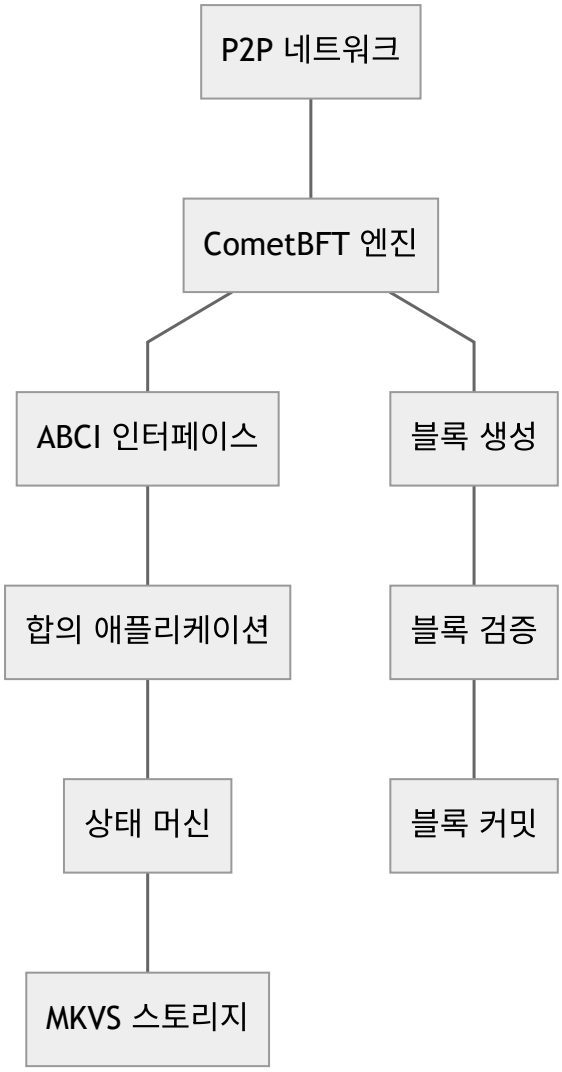
관련 프로세스

- **비콘 서비스**: 에포크 기반 시간 관리와 랜덤성 제공
- **스테이킹 서비스**: 밸리데이터 선정과 보상 분배
- **스케줄러 서비스**: 위원회 구성과 역할 할당
- **루트해시 서비스**: 런타임 상태 커밋 처리

아키텍처

전체 구조

Oasis Core의 합의 프로세스는 CometBFT 엔진을 기반으로 하며, ABCI (Application Blockchain Interface)를 통해 애플리케이션 로직과 분리되어 있습니다.



주요 컴포넌트

CometBFT 엔진

역할: 합의 알고리즘 실행과 P2P 네트워킹 관리

주요 기능:

- 블록 제안과 투표 과정 관리
- 네트워크 통신과 피어 관리
- 합의 상태 머신 실행
- 블록 저장과 동기화

ABCI 애플리케이션

역할: 블록체인 상태 관리와 트랜잭션 처리

주요 기능:

- 트랜잭션 유효성 검증
- 상태 변경 적용
- 쿼리 처리
- 체크포인트 생성

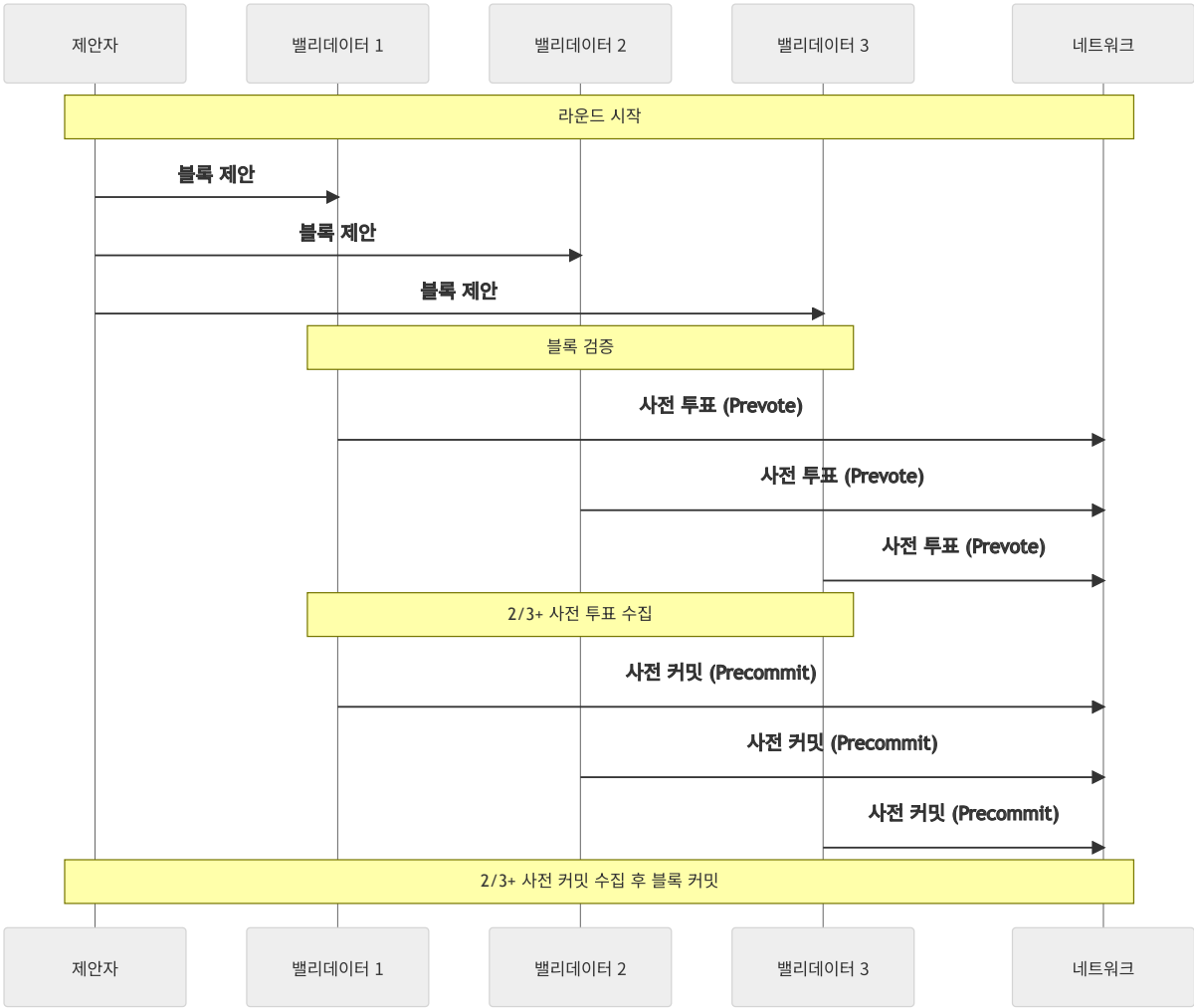
합의 상태 머신

역할: 합의 과정의 각 단계 실행

주요 기능:

- 제안(Propose) 단계 처리
- 사전 투표(Prevote) 단계 처리
- 사전 커밋(Precommit) 단계 처리
- 커밋(Commit) 단계 처리

합의 과정



블록 생성과 검증

블록 생성 과정

1. **제안자 선정**: 라운드 로빈 방식으로 제안자 선정
2. **트랜잭션 수집**: 멤풀에서 유효한 트랜잭션들을 선택
3. **블록 구성**: 헤더, 트랜잭션, 증거 등을 포함한 블록 생성
4. **블록 제안**: 네트워크에 블록 제안 브로드캐스트

```
// 블록 구조 예시
type Block struct {
Height  int64  `json:"height"`
Hash hash.Hash `json:"hash"`
Time time.Time `json:"time"`
StateRoot mkvsNode.Root `json:"state_root"`
Size  uint64  `json:"size,omitempty"`
Meta cbor.RawMessage `json:"meta"`
}
```

블록 검증 과정

1. **기본 검증**: 블록 구조와 서명 유효성 확인
2. **트랜잭션 검증**: 각 트랜잭션의 유효성 검증
3. **상태 검증**: 상태 변경의 정확성 확인
4. **합의 규칙 검증**: 합의 프로토콜 규칙 준수 확인

```
// 블록 검증 함수 예시
func (app *consensusApp) CheckTx(req abci.RequestCheckTx) abci.ResponseCheckTx {
// 트랜잭션 디코딩
```

```go
var  tx transaction.SignedTransaction
if  err := cbor.Unmarshal(req.Tx, &tx);  err != nil {
return abci.ResponseCheckTx{
Code: code.CodeInvalidArgument,
Log: "트랜잭션 디코딩 실패: " + err.Error(),
}
}
// 트랜잭션 유효성 검증
if  err := app.validateTransaction(&tx);  err != nil {
return abci.ResponseCheckTx{
Code: code.CodeInvalidTransaction,
Log: "트랜잭션 검증 실패: " + err.Error(),
}
}
return abci.ResponseCheckTx{Code: code.CodeOK}
}
```

밸리데이터 역할과 책임

밸리데이터의 주요 역할

블록 제안

- 자신의 차례에 새로운 블록 제안

- 유효한 트랜잭션만 포함하여 블록 구성
- 적절한 타이밍에 블록 제안

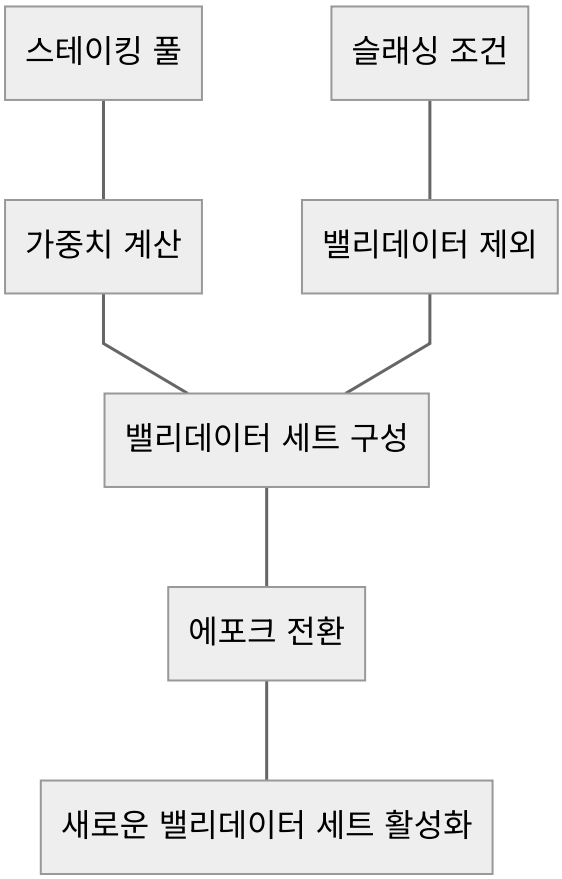
블록 검증

- 제안된 블록의 유효성 검증
- 트랜잭션 실행 결과 확인
- 상태 변경의 정확성 검증

투표 참여

- 사전 투표(Prevote) 단계 참여
- 사전 커밋(Precommit) 단계 참여
- 적절한 타이밍에 투표 제출

밸리데이터 선정 과정



밸리데이터 책임

가용성 유지

- 네트워크에 지속적으로 연결 유지
- 적시에 블록 제안과 투표 참여
- 높은 업타임 유지

보안 관리

- 개인키 안전한 보관
- 이중 서명 방지
- 적절한 네트워크 보안 설정

네트워크 참여

- 거버넌스 제안에 대한 투표
- 네트워크 업그레이드 참여
- 커뮤니티 활동 참여

API 및 인터페이스

합의 백엔드 인터페이스

```
// Backend는 합의 백엔드의 주요 인터페이스입니다
type Backend interface {
// SubmitTx는 서명된 트랜잭션을 제출하고 블록에 포함될 때까지 대기합니다
SubmitTx(ctx context.Context, tx *transaction.SignedTransaction) error
// GetBlock은 특정 높이의 합의 블록을 반환합니다
```

```
GetBlock(ctx context.Context, height  int64) (*Block, error)

// GetLatestHeight는 최신 합의 블록의 높이를 반환합니다

GetLatestHeight(ctx context.Context) (int64, error)

// WatchBlocks는 새로운 블록이 생성될 때마다 알림을 받는 채널을 반환합니다

WatchBlocks(ctx context.Context) (<-chan *Block, pubsub.ClosableSubscription, er

// GetStatus는 현재 합의 상태 개요를 반환합니다

GetStatus(ctx context.Context) (*Status, error)

}
```

ABCI 애플리케이션 인터페이스

```
// Application은 ABCI 애플리케이션 인터페이스입니다

type Application interface {

// CheckTx는 트랜잭션의 유효성을 검증합니다

CheckTx(abci.RequestCheckTx) abci.ResponseCheckTx

// DeliverTx는 트랜잭션을 실행하고 상태를 업데이트합니다

DeliverTx(abci.RequestDeliverTx) abci.ResponseDeliverTx

// BeginBlock은 새로운 블록 처리를 시작합니다

BeginBlock(abci.RequestBeginBlock) abci.ResponseBeginBlock

// EndBlock은 블록 처리를 완료합니다

EndBlock(abci.RequestEndBlock) abci.ResponseEndBlock

// Commit은 상태 변경을 커밋합니다

Commit() abci.ResponseCommit
```

```
}
```

gRPC API

합의 서비스 정의

```
service ConsensusService {
// GetBlock은 특정 높이의 블록을 조회합니다
rpc  GetBlock(GetBlockRequest) returns (GetBlockResponse);
// GetLatestHeight는 최신 블록 높이를 조회합니다
rpc  GetLatestHeight(GetLatestHeightRequest) returns (GetLatestHeightResponse);
// SubmitTx는 트랜잭션을 제출합니다
rpc  SubmitTx(SubmitTxRequest) returns (SubmitTxResponse);
// GetStatus는 합의 상태를 조회합니다
rpc  GetStatus(GetStatusRequest) returns (GetStatusResponse);
}
```

메시지 타입

```
message GetBlockRequest {
int64  height = 1; // 블록 높이 (0은 최신 블록)
}

message GetBlockResponse {
```

```
Block  block = 1; // 블록 정보
}

message SubmitTxRequest {
bytes  signed_tx = 1; // 서명된 트랜잭션
}

message SubmitTxResponse {
// 응답은 비어있음 (에러 시 gRPC 에러 반환)
}
```

설정 및 구성

합의 설정 구조

```yaml
consensus:
# CometBFT 엔진 설정
backend: cometbft
# 네트워크 설정
listen_address: "tcp://0.0.0.0:26656"
external_address: "tcp://node.example.com:26656"
# 타이밍 설정
timeout_commit: "1s"
```

```yaml
skip_timeout_commit: false
empty_block_interval: "0s"
# P2P 설정
p2p:
max_num_inbound_peers: 40
max_num_outbound_peers: 10
persistent_peers: []
unconditional_peers: []
# 상태 동기화 설정
state_sync:
enabled: false
trust_height: 0
trust_hash: ""
```

주요 설정 옵션

옵션 이름	타입	기본값	설명
`backend`	string	“cometbft”	합의 백엔드 종류
`listen_address`	string	“tcp://0.0.0.0:26656”	P2P 리스닝 주소
`timeout_commit`	duration	“1s”	커밋 후 대기 시간
`empty_block_interval`	duration	“0s”	빈 블록 생성 간격
`p2p.max_num_inbound_peers`	int	40	최대 인바운드 피어 수
`p2p.max_num_outbound_peers`	int	10	최대 아웃바운드 피어 수

환경 변수

| 환경 변수 | 설명 | 예시 |

|---|---|---|

| `OASIS_CONSENSUS_BACKEND` | 합의 백엔드 지정 | `cometbft` |

| `OASIS_CONSENSUS_LISTEN_ADDRESS` | P2P 리스닝 주소 | `tcp://0.0.0.0:26656` |

| `OASIS_CONSENSUS_LOG_LEVEL` | 합의 로그 레벨 | `info`, `debug`, `error` |

코드 예제

기본 사용법

```go
package main


import (
"context"
"fmt"
"log"
"github.com/oasisprotocol/oasis-core/go/consensus/api"
"github.com/oasisprotocol/oasis-core/go/consensus/api/transaction"
)


func main() {
// 합의 백엔드 연결
consensus, err := api.NewConsensusClient("unix:/tmp/oasis-node-consensus.sock")
```

```go
if err != nil {
log.Fatal("합의 클라이언트 생성 실패:", err)
}
ctx := context.Background()
// 최신 블록 높이 조회
height, err := consensus.GetLatestHeight(ctx)
if err != nil {
log.Fatal("최신 높이 조회 실패:", err)
}
fmt.Printf("최신 블록 높이: %d\n", height)
// 특정 블록 조회
block, err := consensus.GetBlock(ctx, height)
if err != nil {
log.Fatal("블록 조회 실패:", err)
}
fmt.Printf("블록 해시: %s\n", block.Hash)
fmt.Printf("블록 시간: %s\n", block.Time)
}
```

트랜잭션 제출

```go
// 트랜잭션 생성 및 제출
func submitTransaction(consensus api.Backend, signer signature.Signer) error {
```

```go
ctx := context.Background()

// 트랜잭션 생성

tx := &transaction.Transaction{

Nonce: 1,

Fee: transaction.NewFee(1000, 0),

Method: "staking.Transfer",

Body: &staking.Transfer{

To: staking.NewAddress(recipientID),

Amount: *quantity.NewFromUint64(1000000),

},

}

// 트랜잭션 서명

signedTx, err := transaction.Sign(signer, tx)

if err != nil {

return fmt.Errorf("트랜잭션 서명 실패: %w", err)

}

// 트랜잭션 제출

if err := consensus.SubmitTx(ctx, signedTx); err != nil {

return fmt.Errorf("트랜잭션 제출 실패: %w", err)

}

fmt.Println("트랜잭션이 성공적으로 제출되었습니다")

return nil
```

```
}
```

블록 모니터링

```
// 새로운 블록 모니터링
func watchBlocks(consensus api.Backend) error {
ctx := context.Background()
// 블록 스트림 구독
blockCh, sub, err := consensus.WatchBlocks(ctx)
if err != nil {
return fmt.Errorf("블록 구독 실패: %w", err)
}
defer sub.Close()
fmt.Println("새로운 블록을 모니터링합니다...")
for {
select {
case block := <-blockCh:
fmt.Printf("새로운 블록: 높이=%d, 해시=%s, 시간=%s\n",
block.Height, block.Hash, block.Time)
case <-ctx.Done():
return ctx.Err()
}
}
```

```
}
```

CLI 사용 예제

```
# 합의 상태 확인
oasis-node consensus status

# 특정 높이의 블록 조회
oasis-node consensus show-block --height 12345

# 최신 블록 정보 조회
oasis-node consensus show-block --height 0

# 트랜잭션 제출
oasis-node consensus submit-tx --transaction tx.json

# 합의 파라미터 조회
oasis-node consensus show-parameters --height 0
```

모니터링 및 메트릭

주요 메트릭

| 메트릭 이름 | 타입 | 설명 |

|---|---|---|

| `oasis_consensus_blocks_total` | Counter | 총 처리된 블록 수 |

| `oasis_consensus_transactions_total` | Counter | 총 처리된 트랜잭션 수 |

| `oasis_consensus_block_interval_seconds` | Histogram | 블록 생성 간격 |

| `oasis_consensus_round_duration_seconds` | Histogram | 합의 라운드 소요 시간 |

| `oasis_consensus_validators_total` | Gauge | 현재 밸리데이터 수 |

| `oasis_consensus_voting_power_total` | Gauge | 총 투표권 |

헬스 체크

```
# HTTP 헬스 체크 엔드포인트
curl http://localhost:9001/health

# 응답 예시
{
"status": "ready",
"latest_height": 12345,
"latest_hash": "a1b2c3d4e5f6...",
"latest_time": "2024-01-01T12:00:00Z",
"is_validator": true,
"is_syncing": false
}
```

로그 모니터링

```
# 합의 관련 로그 확인
journalctl -u oasis-node -f | grep consensus

# 에러 로그 필터링
journalctl -u oasis-node --since "1 hour ago" | grep ERROR | grep consensu

# 특정 높이 관련 로그 확인
journalctl -u oasis-node | grep "height=12345"
```

문제 해결

일반적인 문제

문제 1: 노드가 합의에 참여하지 않음

증상: 노드가 블록을 제안하거나 투표하지 않음

원인:

- 밸리데이터 세트에 포함되지 않음
- 네트워크 연결 문제
- 시간 동기화 문제

해결 방법:

1. 밸리데이터 등록 상태 확인

```
oasis-node registry node list --verbose
```

2. 네트워크 연결 상태 확인

```
oasis-node consensus status | grep  peers
```

3. 시스템 시간 동기화 확인

```
timedatectl status
```

문제 2: 블록 동기화 지연

증상: 노드가 최신 블록을 따라가지 못함

원인:

- 네트워크 대역폭 부족
- 디스크 I/O 성능 문제
- 피어 연결 부족

해결 방법:

1. 네트워크 대역폭 확인
2. 디스크 성능 모니터링
3. 더 많은 피어 추가

```
consensus:
p2p:
persistent_peers:
- "peer1@node1.example.com:26656"
- "peer2@node2.example.com:26656"
```

문제 3: 트랜잭션 처리 지연

증상: 제출된 트랜잭션이 블록에 포함되지 않음

원인:

- 가스 가격 부족
- 트랜잭션 유효성 문제
- 멤풀 포화 상태

해결 방법:

1. 가스 가격 확인 및 조정
2. 트랜잭션 유효성 검증
3. 멤풀 상태 모니터링

```
oasis-node consensus unconfirmed-transactions
```

로그 분석

```
# 합의 에러 로그 확인
grep "consensus.*ERROR" /var/log/oasis/node.log

# 투표 관련 로그 확인
grep "vote\|prevote\|precommit" /var/log/oasis/node.log

# 블록 제안 관련 로그 확인
grep "propose\|proposal" /var/log/oasis/node.log
```

```
# 네트워크 연결 문제 확인
grep "peer\|connection" /var/log/oasis/node.log
```

성능 튜닝

네트워크 최적화

```yaml
consensus:
p2p:
send_rate: 5120000  # 5MB/s
recv_rate: 5120000  # 5MB/s
max_packet_msg_payload_size: 1024
```

타이밍 최적화

```yaml
consensus:
timeout_commit: "1s"
skip_timeout_commit: false
empty_block_interval: "0s"
```

참조

관련 문서

- 비콘 서비스
- 스테이킹 서비스
- 스케줄러 서비스
- 루트해시 서비스
- 아키텍처 개요

소스 코드

- 합의 API
- CometBFT 구현
- ABCI 애플리케이션

외부 리소스

- CometBFT 공식 문서
- ABCI 스펙
- 비잔틴 장애 허용 합의

마지막 업데이트: 2024년 1월

작성자: Oasis Core 문서화 팀

검토자: 합의 개발팀